

```

In [6]: import numpy as np
import matplotlib.pyplot as plt

m = 1.0
k = 1.0
omega = np.sqrt(k / m)
A = 1.0
dt = 0.1
t_max = 50.0
steps = int(t_max / dt)

x = [A]
v = [0.0]

for i in range(steps):
    x_new = x[-1] + v[-1] * dt
    v_new = v[-1] - (k / m) * x[-1] * dt
    x.append(x_new)
    v.append(v_new)

t = np.linspace(0, t_max, steps + 1)
x_analytical = A * np.cos(omega * t)

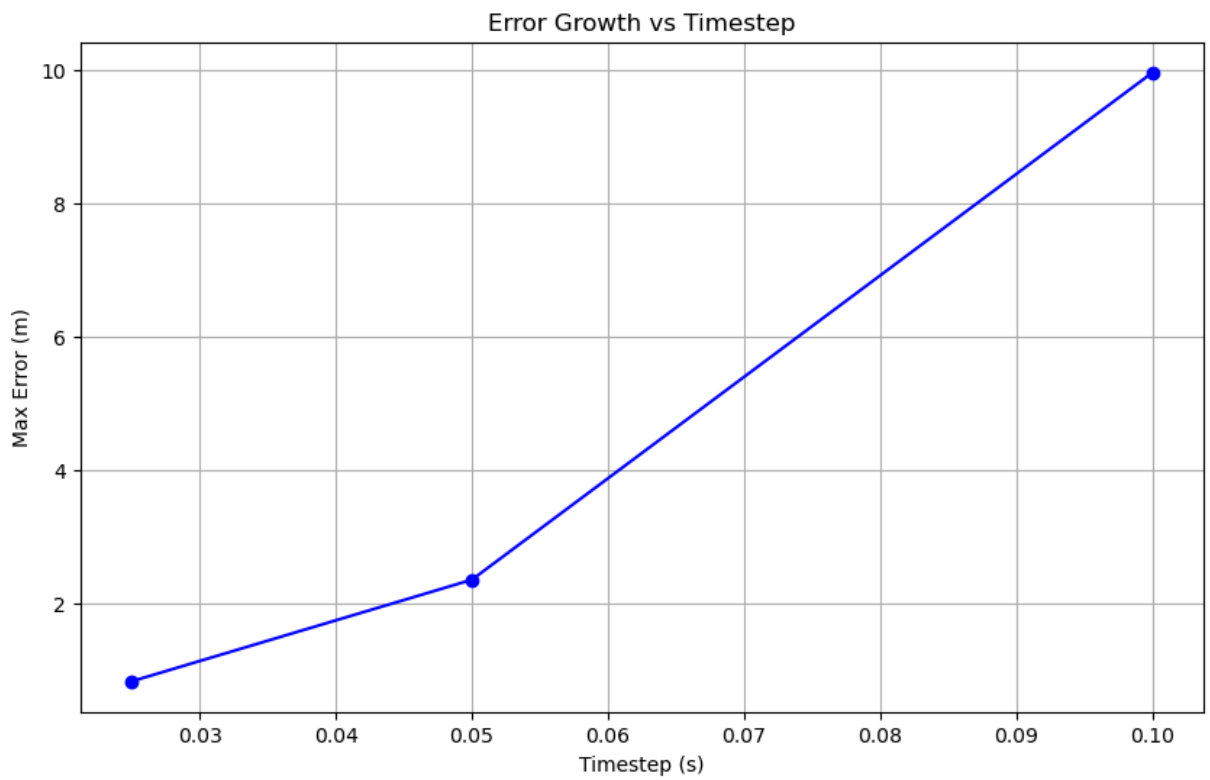
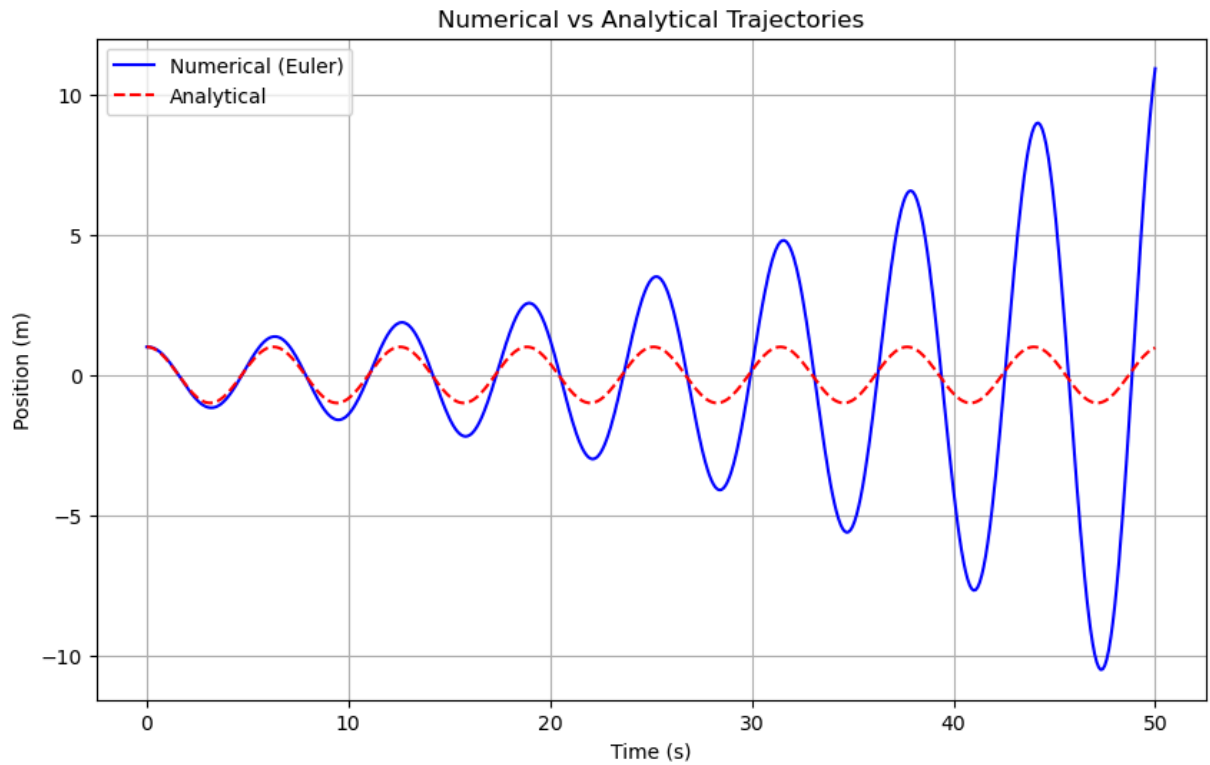
plt.figure(figsize=(10, 6))
plt.plot(t, x, 'b-', label='Numerical (Euler)')
plt.plot(t, x_analytical, 'r--', label='Analytical')
plt.xlabel('Time (s)')
plt.ylabel('Position (m)')
plt.title('Numerical vs Analytical Trajectories')
plt.legend()
plt.grid(True)
plt.show()

dt_values = [0.1, 0.05, 0.025]
errors = []
for dt in dt_values:
    steps = int(t_max / dt)
    x = [A]
    v = [0.0]
    for i in range(steps):
        x_new = x[-1] + v[-1] * dt
        v_new = v[-1] - (k / m) * x[-1] * dt
        x.append(x_new)
        v.append(v_new)
    t = np.linspace(0, t_max, steps + 1)
    x_analytical = A * np.cos(omega * t)
    error = np.max(np.abs(np.array(x) - x_analytical))
    errors.append(error)

plt.figure(figsize=(10, 6))
plt.plot(dt_values, errors, 'bo-')
plt.xlabel('Timestep (s)')
plt.ylabel('Max Error (m)')
plt.title('Error Growth vs Timestep')

```

```
plt.grid(True)  
plt.show()
```



In [ ]: